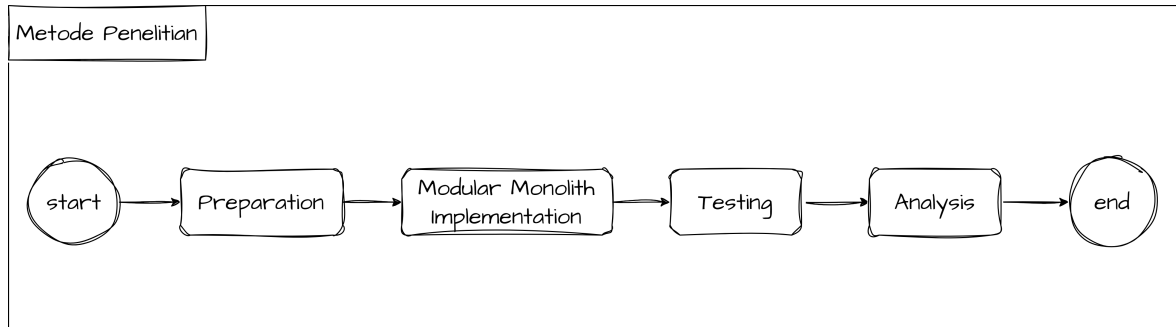




BAB III

METODE PENELITIAN



Gambar 3.1. Diagram Metode Penelitian

Secara umum, penelitian ini dilakukan melalui beberapa tahapan utama yang ditunjukkan pada Gambar 3.1. Tahapan tersebut dimulai dari *Preparation*, yaitu tahap persiapan yang mencakup *feature selection* dan *tools configuration*. Selanjutnya adalah tahap *Modular Monolith Implementation*, di mana sistem dikembangkan menggunakan pendekatan arsitektur *modular monolith*. Setelah implementasi selesai, dilakukan tahap *Testing* untuk menguji sistem dengan metrik dan alat yang telah ditentukan. Hasil pengujian kemudian dianalisis pada tahap *Analysis* untuk mengevaluasi dampak modularisasi terhadap kualitas sistem.

3.1 *Preparation*

3.1.1 *Feature Selection*

Dalam tahap *Feature Selection*, pemilihan fitur difokuskan pada peran *Admin*. Hal itu dikarenakan cakupannya yang mencakup seluruh aspek aplikasi, termasuk fitur yang dapat diakses maupun fitur yang dibatasi untuk peran *Student* dan *Teacher*. Dengan memilih fitur dari peran *Admin*, evaluasi dapat mencerminkan interaksi sistem secara menyeluruh, termasuk berbagai akses ke modul berbeda serta alur kerja yang kompleks. Hal ini memastikan bahwa fitur yang diuji dapat merepresentasikan keseluruhan fungsionalitas aplikasi dan tidak terbatas pada subset tertentu dari pengguna.

Proses seleksi fitur dilakukan dengan mempertimbangkan beberapa aspek utama, yaitu dampak atau urgensi fitur, jumlah domain entitas yang terlibat, jumlah modul yang diakses, serta jumlah proses yang dibutuhkan dalam eksekusi fitur tersebut (Goncalves, dkk., 2021). Urgensi fitur terhadap alur sistem dilihat dari tingkat independensi entitas dalam basis data. Fitur-fitur yang bersifat urgen dan independen, seperti operasi *CREATE* pada entitas dasar yang tidak memiliki ketergantungan ke entitas lain saat proses *insertion*, menjadi prioritas utama.

Tabel 3.1. Daftar Fitur Urgen pada Aplikasi Tahsin App

No	Fitur
1	<i>Register</i> (sebagai representasi <i>CREATE - User</i>)
2	<i>CREATE - Bank Account</i>
3	<i>CREATE - Transaction Status</i>
4	<i>CREATE - Transaction Type</i>
5	<i>CREATE - Level</i>
6	<i>CREATE - Student Status</i>
7	<i>CREATE - Day</i>
8	<i>CREATE - Batch</i>
9	<i>CREATE - Class Price</i>

Tabel 3.1 merupakan contoh dari fitur-fitur yang termasuk fitur urgen. Fokus pada fitur-fitur urgen bertujuan untuk mengevaluasi performa sistem terhadap skenario dasar yang mendasari fitur-fitur turunan lainnya. Selanjutnya fitur-fitur yang tidak termasuk dalam kategori urgen akan dievaluasi kompleksitasnya menggunakan tiga parameter utama, yaitu jumlah entitas domain yang terlibat, jumlah modul yang diakses, dan jumlah proses dalam satu alur fitur. Dengan pendekatan ini, fitur yang dipilih dapat memberikan gambaran yang lebih realistis terhadap kondisi sistem saat beroperasi. Selain itu, strategi ini memastikan bahwa pengujian tidak hanya mencerminkan satu skenario spesifik, tetapi mampu mengevaluasi berbagai kemungkinan interaksi yang terjadi dalam aplikasi secara keseluruhan.

3.1.2 Tools Configuration

Untuk memastikan pengujian berjalan optimal, beberapa alat dikonfigurasi dengan parameter yang sesuai dengan kebutuhan penelitian. Postman digunakan untuk mengonfigurasi *endpoint* fitur *Admin* dengan metode utama *GET*, *POST*, dan *PATCH*. Konfigurasi ini memungkinkan simulasi interaksi pengguna terhadap sistem secara menyeluruh, sehingga setiap skenario dapat diuji dengan konsistensi yang tinggi.

Analisis kualitas kode dilakukan dengan SonarQube, yang dikonfigurasi agar lebih sensitif terhadap perubahan arsitektur. Penyesuaian dilakukan dengan *tune-up* parameter, termasuk perubahan dalam perhitungan *development cost* agar lebih realistis berdasarkan estimasi waktu penulisan kode dalam kondisi nyata. Selain itu, beberapa aturan terkait *maintainability* diaktifkan dan dinonaktifkan untuk memastikan bahwa analisis kode lebih akurat dalam mendeteksi perubahan struktural. Kategori aturan yang dinonaktifkan adalah aturan yang berstatus *deprecated*; aturan mengenai *constraint* penulisan seperti penulisan *logical* dan *comparison operator*; dan beberapa aturan yang berfokus pada *line management*. Kategori aturan yang diaktifkan adalah aturan yang memiliki *height severity*; aturan yang bertujuan untuk deteksi potensi *bug*; dan aturan yang berfokus pada penataan logika percabangan, perulangan, dan pengaturan *class*. Tabel 3.2 menunjukkan perbandingan antara konfigurasi awal dan hasil penyesuaian aturan pada SonarQube.

Tabel 3.2. Perbandingan Konfigurasi SonarQube Sebelum dan Setelah *Tune-Up*

<i>Item</i>	<i>Default</i>	<i>Setelah Tune-Up</i>
Estimasi Beban Pengembangan (<i>Development Cost</i>)	30 menit	3 menit
Jumlah Aturan Aktif (<i>Active Rules</i>)	182	109
Jumlah Aturan Nonaktif (<i>Inactive Rules</i>)	75	148

Untuk pengujian performa, JMeter dikonfigurasi dengan penambahan *header* dan variabel global yang digunakan oleh beberapa *request*. *Test script* yang dibuat mengikuti skenario pengujian yang telah dirancang pada tahap *writing test scenarios*. Dengan pendekatan ini, setiap iterasi pengujian menghasilkan data yang konsisten dan dapat dibandingkan secara objektif.

3.1.3 Batasan Studi Kasus

Penelitian ini hanya berfokus pada implementasi *backend* tanpa mencakup pengembangan *frontend* atau integrasi dengan aplikasi eksternal. Selain itu, studi dilakukan pada aplikasi dengan skala kecil-menengah, di mana jumlah pengguna diperkirakan tidak lebih dari 100 orang. Dalam pengembangannya, Tahsin App hanya menggunakan *library* dari ekosistem JavaScript, memastikan konsistensi teknologi yang digunakan dalam sistem. Terakhir, pengujian dilakukan dalam lingkungan *localhost*, tanpa melibatkan *deployment* ke *cloud* atau produksi untuk meminimalkan kompleksitas infrastruktur.

3.1.4 Hardware Specification

Tabel 3.3 menunjukkan dua perangkat yang digunakan dalam pengujian. Server 1 digunakan untuk menjalankan seluruh jenis pengujian, baik pengujian statik menggunakan SonarQube maupun pengujian beban menggunakan JMeter. Server 2 hanya digunakan untuk pengujian beban (*load test*) sebagai bagian dari skenario *vertical scaling*, di mana perangkat dengan spesifikasi lebih tinggi digunakan untuk melihat dampak peningkatan kapasitas perangkat keras terhadap performa aplikasi. Perbedaan spesifikasi perangkat, seperti CPU dan memori, memberikan konteks dalam interpretasi hasil pengujian performa sistem yang dilakukan.

Tabel 3.3. Spesifikasi Perangkat Keras Untuk Pengujian

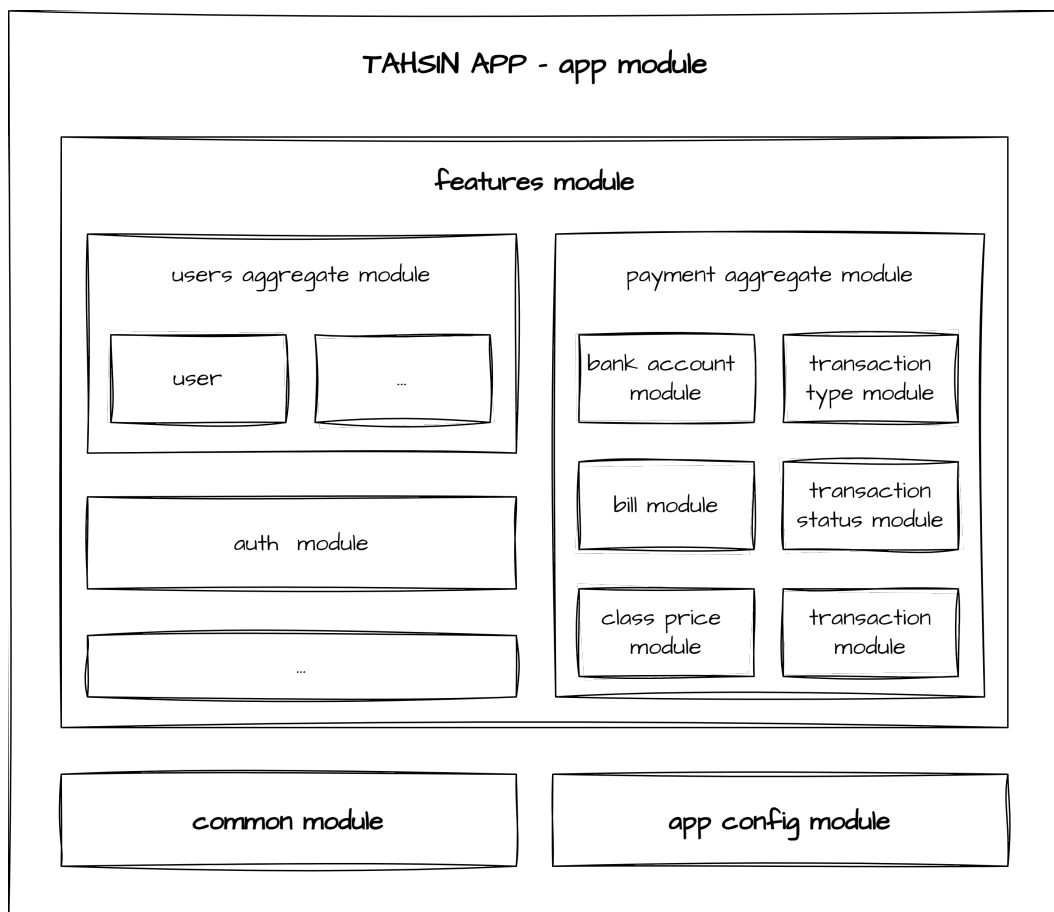
Spesifikasi	Server 1 (Lenovo Ideapad Gaming 3 Gen 6)	Server 2 (Acer Nitro)
Pengujian <i>static test</i>	✓	-
Pengujian <i>load test</i>	✓	✓
<i>Operating System</i> (OS)	Windows 11 Home	Windows 11 Pro
CPU	Intel Core i5-11300H (3.10GHz, 8 CPUs)	Intel Core i9-12900H (2.90GHz)
Memory	16 GB	32 GB

3.2 Implementation

3.2.1 Design Architecture

Tahap awal implementasi dimulai dengan pendefinisian batasan setiap *layer* aplikasi, yang bertujuan untuk memperjelas ruang lingkup masing-masing *layer* sehingga *separation of concern* tetap terjaga. Dengan pendekatan ini, setiap bagian dari sistem memiliki tanggung jawab yang spesifik, mengurangi kompleksitas dan meningkatkan efisiensi dalam pengembangan (Goncalves, dkk., 2021; Oumoussa dan Saidi, 2025). Selain itu, prinsip “*share as little as possible*” diterapkan untuk memastikan bahwa antar-komponen hanya berbagi informasi yang benar-benar diperlukan, sehingga isolasi antar modul tetap terjaga (Waseem, dkk., 2021).

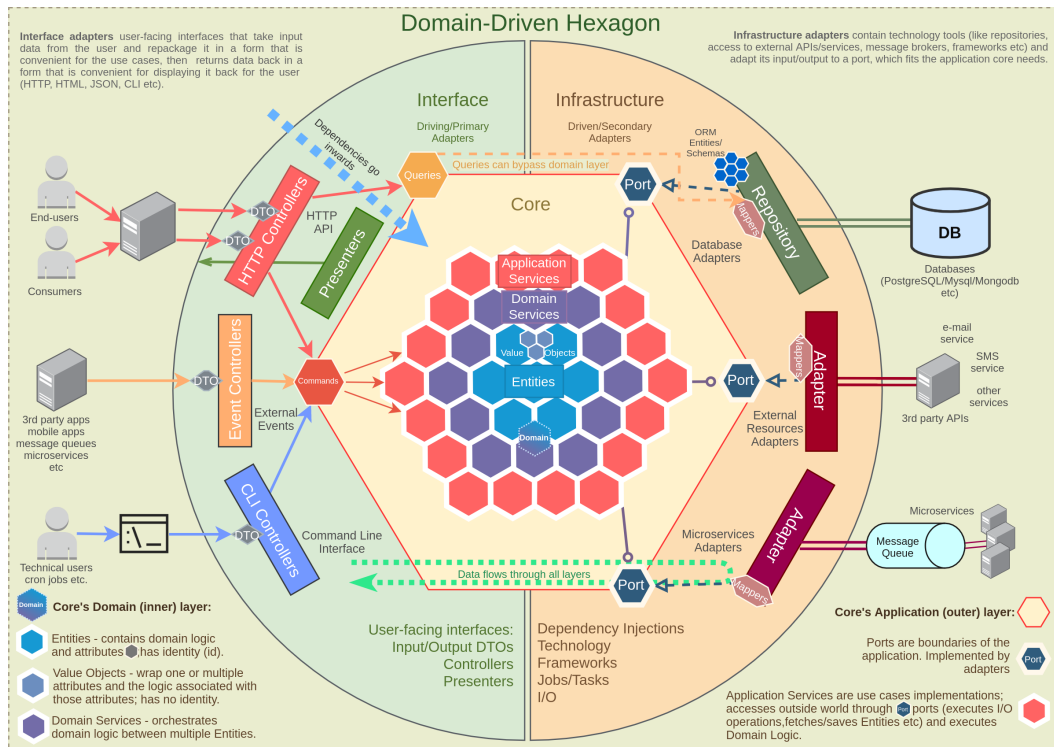
Dalam proses definisi layanan (*services*), arsitektur aplikasi dipecah menjadi satuan layanan yang mengikuti prinsip *Service-Oriented Architecture (SOA)*. Pemisahan ini dilakukan berdasarkan *business capability* dan diperkuat dengan pendekatan *Domain-Driven Design (DDD)*, sehingga setiap modul tetap mengikuti kedekatan relasi domain secara bisnis (Waseem, dkk., 2021; Fritzsch, 2024; Oumoussa dan Saidi, 2025). Dengan metode ini, modularisasi tidak hanya dilakukan berdasarkan fungsi teknis, tetapi juga mempertimbangkan kebutuhan bisnis yang mendasarinya.



Gambar 3.2. Desain Modular Monolith pada Tahsin App

Implementasi arsitektur *modular monolith* dalam Tahsin App seperti yang terlihat pada Gambar

3.2 dilakukan dengan membungkus seluruh modul fitur ke dalam satu modul utama yang disebut *AppModule*. Modul ini menjadi titik pusat integrasi berbagai fitur. Kemudian *services* lainnya dipecah dan dikelompokkan berdasarkan kedekatan *business*-nya. Segala hal yang berkaitan dengan *payment* dikelompokkan ke dalam *payment aggregate module*, begitu seterusnya.



Gambar 3.3. Desain *Hexagonal Architecture* pada Setiap Modul (Doshi, 2024)

Struktur ini didukung oleh pendekatan *Hexagonal Architecture* sebagai kerangka implementasi internal setiap modul. Seperti ditunjukkan pada Gambar 3.3, arsitektur ini memisahkan logika domain dari infrastruktur teknis dengan mengandalkan konsep *Port and Adapter*. Interaksi keluar-masuk modul dikendalikan oleh *ports* yang diidentifikasi dengan *provider token* untuk menjamin isolasi logika bisnis dan fleksibilitas pengujian.

3.2.2 Setup Configuration

Langkah berikutnya dalam implementasi adalah *setup* konfigurasi, yang mencakup instalasi *framework*, pengaturan *version control*, dan persiapan lingkungan kerja. *Framework* yang digunakan adalah NestJS, karena secara *native* mendukung modularisasi yang diperlukan dalam arsitektur *modular monolith*. Untuk memastikan stabilitas dan konsistensi dalam pengembangan kode, digunakan WebStorm IDE, yang dikenal dengan dukungannya terhadap standar *CLEAN Code*. Selain itu, dilakukan *setup linting* dan indentasi sesuai dengan aturan dari SonarQube, guna memastikan standar penulisan kode yang seragam pada kedua arsitektur, baik *monolith* maupun *modular monolith*. Tahap ini juga mencakup *install dependencies* menggunakan package npm, di mana seluruh dependensi yang digunakan dalam versi *monolithic* dipasang kembali untuk memastikan keselarasan antara kedua arsitektur yang dibandingkan.

Tabel 3.4. Struktur *Layer* Berdasarkan *Hexagonal Architecture*

Layer	Komponen
<i>Shared Layer</i>	<i>Constants, Decorators, Interceptors, Pipes, Services, Types, Utils</i>
<i>Domain Layer</i>	<i>Entities, Port</i>
<i>Application Layer</i>	<i>Service, Usecase</i>
<i>Infrastructure Layer</i>	<i>Mapper, Persistence / Repository</i>
<i>Interface Layer</i>	<i>Controllers, Schemas and DTOs, Mapper, Interceptor</i>

3.2.3 Set Structures of Directory

Dalam tahap ini, struktur folder disesuaikan dengan desain arsitektur *hexagonal* dan pendekatan berbasis *services*. Pendekatan *hexagonal architecture* digunakan untuk memisahkan domain bisnis dari aspek teknis aplikasi. Detail pemisahan *layer* dan komponennya dapat dilihat pada Table 3.4. Sementara untuk detail struktur foldernya dapat dilihat pada Table 3.5. Setiap *layer* membutuhkan mekanisme atau komponen khusus untuk saling berkomunikasi, kecuali pada *shared layer* yang memang didedikasikan untuk menyediakan fungsi-fungsi yang akan digunakan di semua lapisan. Sesuai aturan *hexagonal architecture* maka *core layer* yang terdiri dari *domain* dan *application layer* harus melalui *port* dan *adapter* untuk bisa berkomunikasi dengan *infrastructure* dan *interface layer*. Dengan demikian, logika bisnis yang berada pada *core layer* tetap independen dan mudah diuji atau diadaptasi tanpa ketergantungan langsung pada infrastruktur teknis tertentu (Michail, 2023).

Tabel 3.5. Struktur Direktori Aplikasi

Folder	Deskripsi
common/	Berisi kode yang bersifat <i>shared</i> dan digunakan lintas modul, seperti <i>interceptors</i> , <i>decorators</i> , <i>services</i> , <i>pipes</i> , serta <i>types</i> . Folder ini membantu menghindari duplikasi dan menjaga standar antar bagian aplikasi.
app/	Menjadi pusat konfigurasi dan integrasi aplikasi, termasuk inisialisasi <i>global modules</i> , pengaturan dependensi, dan pengelolaan berkas <code>appModule.ts</code> .
features/	Menyimpan seluruh fitur yang ada dalam aplikasi, seperti <i>auth</i> , <i>users</i> , <i>registration</i> , <i>courses</i> , <i>scheduling</i> , dan lainnya. Masing-masing fitur memiliki struktur heksagonal tersendiri yang terdiri dari <i>domain</i> , <i>application</i> , <i>infrastructure</i> , <i>interface/presentation</i> , dan <i>module</i> . Folder ini mencerminkan penerapan prinsip <i>separation of concern</i> dan mempermudah pengelolaan fitur secara terisolasi namun tetap terintegrasi dalam arsitektur <i>modular monolith</i> .

3.2.4 Common Directory

Direktori ini berisi berbagai komponen umum yang digunakan lintas modul, seperti konstanta, layanan umum, dekorator, serta utilitas bantu. Tujuannya adalah untuk menjaga konsistensi dan menghindari duplikasi fungsi di seluruh aplikasi. Detail isi dari folder `common/` dapat dilihat pada Tabel 3.6.

Tabel 3.6. Isi Folder `common/` dan Fungsinya

Folder	Berkas / Folder	Deskripsi
constants	<code>const.ts</code> <code>key.ts</code> <code>provider.token.ts</code>	Variabel global seperti <code>BASE_PATH</code> . Metadata seperti <code>IS_PUBLIC_KEY</code> , <code>X_API_TOKEN</code> . Token untuk kebutuhan <i>dependency injection</i> .
decorators	<code>public.decorator.ts</code> <code>user.decorator.ts</code>	Menandai rute sebagai publik agar bisa diakses tanpa autentikasi. Menyisipkan data <i>user</i> dari request untuk modul yang membutuhkannya.
interceptors	<code>logging.interceptor.ts</code>	Mencatat seluruh request dan response untuk audit dan <i>debugging</i> .
pipes	<code>validation.pipe.ts</code>	Validasi data yang masuk dengan pendekatan berbasis skema.
services	<code>crypto.service.ts</code> <code>database.service.ts</code> <code>logger.service.ts</code> <code>validation.service.ts</code>	<i>Hashing</i> menggunakan <code>argon2</code> dan pembuatan token dengan <code>UUID</code> . Membungkus Prisma <i>instance</i> sebagai <i>singleton service</i> . <i>Logger</i> berbasis konsol. Fungsi validasi data untuk penggunaan global.
types	<code>enums</code> <code>usecase.ts</code>	Menyimpan enumerasi seperti status kehadiran. Mendefinisikan struktur standar untuk <i>usecase</i> .
utils	<code>time.service.ts</code>	Fungsi bantu untuk manipulasi dan format waktu.

3.2.5 Core Layer Directories

Direktori *core layer* terdiri dari dua lapisan utama, yaitu *domain* dan *application*, yang masing-masing memiliki peran berbeda namun saling melengkapi. *Domain layer* berfungsi sebagai inti dari domain bisnis dengan mendefinisikan struktur data serta antarmuka abstrak untuk berinteraksi dengan

infrastruktur, sehingga logika domain tetap terisolasi dari detail teknis. Sementara itu, *application layer* bertugas mengelola alur proses dan logika bisnis, dengan peran sebagai penghubung antara domain dan infrastruktur melalui koordinasi berbagai layanan dan aturan bisnis yang berlaku. Detail deskripsi dan isi dari *core layer* dapat dilihat pada Tabel 3.7.

Tabel 3.7. Struktur Lapisan Domain dan *Application*

<i>Layer</i>	Folder	Deskripsi
<i>Domain Layer</i>	entities	Mendefinisikan struktur dan atribut dari entitas bisnis utama, termasuk validasi dan fungsi bantu.
	port	Abstraksi kontrak dalam bentuk <i>interface</i> untuk <i>infrastructure layer</i> , memudahkan pergantian implementasi di masa depan.
<i>Application Layer</i>	services	Berisi logika spesifik sesuai domain, seperti validasi bisnis dan kalkulasi data.
	usecases	Bertindak sebagai <i>orchestrator</i> yang menggabungkan berbagai <i>service</i> untuk mengeksekusi satu fitur secara <i>end-to-end</i> .

3.2.6 *Infrastructure Directory*

Lapisan *infrastructure* bertugas untuk menyediakan implementasi teknis dari *port* yang telah didefinisikan di *domain layer*. Struktur direktori pada lapisan *infrastructure* dijelaskan secara rinci pada Tabel 3.8, yang mencakup fungsi utama seperti pemetaan data dan implementasi akses *database* menggunakan Prisma.

Tabel 3.8. Struktur Folder Infrastruktur

Folder	Deskripsi
mapper	Berisi <i>mapper</i> yang bertugas untuk mengubah data dari bentuk <i>database</i> menjadi <i>domain entity</i> dan sebaliknya.
persistence	Berisi implementasi dari <i>repository port</i> menggunakan Prisma sebagai ORM, sehingga tiap modul memiliki cara standar dalam mengakses <i>database</i> .

3.2.7 *Interface Directory*

Direktori ini berisi komponen yang berhubungan langsung dengan komunikasi eksternal, baik dengan pengguna maupun sistem lain. Melalui RESTful API, direktori ini menangani proses *request* dan *response*, validasi data, serta transformasi antar format data. Struktur direktori yang membentuk lapisan *interface* dijelaskan lebih lanjut pada Tabel 3.9, yang memperlihatkan tanggung jawab masing-masing folder dalam menangani permintaan klien, validasi data, serta pengelolaan proses komunikasi dengan lapisan aplikasi.

Tabel 3.9. Struktur Direktori *Interface Layer*

Folder	Deskripsi
controllers	Menerima dan menangani <i>request</i> dari klien, serta mengarahkan ke <i>usecase</i> sesuai kebutuhan.
schemas & DTOs	Menangani validasi dan transformasi data dari dan ke format <i>domain entity</i> .
interceptors	Digunakan untuk pengelolaan <i>request-response</i> seperti <i>logging</i> , transformasi, <i>intercept</i> , dan filter data.

3.3 Testing

Untuk memahami performa dari kedua arsitektur, yaitu *monolith* dan *modular monolith*, dilakukan serangkaian pengujian guna menilai efektivitas serta efisiensi masing-masing pendekatan dalam menangani beban sistem. Pengujian dilakukan menggunakan lingkungan, *tools*, konfigurasi, dan skenario pengujian yang identik, serta *test script* yang seragam. Dengan pendekatan ini, setiap hasil yang diperoleh dapat dibandingkan secara objektif tanpa adanya variabel eksternal yang memengaruhi performa sistem. Hasil dari pengujian ini nantinya akan dianalisis lebih lanjut pada tahap berikutnya, memberikan wawasan yang lebih dalam mengenai keunggulan serta *trade-off* dari masing-masing arsitektur.

3.3.1 API Testing

Pengujian API dilakukan berdasarkan dokumentasi resmi Tahsin App yang memuat spesifikasi *endpoint*, metode HTTP, serta format *request-response*. Dokumentasi ini menjadi acuan utama dalam memastikan bahwa setiap permintaan yang dikirimkan menghasilkan respons yang sesuai dengan spesifikasi sistem. Proses pengujian dilakukan menggunakan Postman (Kore, dkk., 2022), sebuah alat bantu yang memungkinkan pengiriman permintaan ke API dan observasi langsung terhadap responsnya. Dengan Postman, penulis dapat memverifikasi bahwa data yang dikirim dan diterima sudah sesuai, serta seluruh fungsi API berjalan sebagaimana mestinya. Seluruh *endpoint* diuji berdasarkan peran *Admin* menggunakan metode *GET*, *POST*, dan *PATCH*. Pengujian ini bertujuan untuk mengevaluasi keandalan sistem dalam menangani berbagai jenis permintaan, memvalidasi skema data, serta memastikan tidak terdapat *error* yang mengganggu fungsionalitas API.

3.3.2 Software Quality Testing

Tabel 3.10. Pengujian Kode Statis (*Static Code Test*)

Quality / Metrics	Tools	Parameter
Maintainability	SonarQube	Maintainability Rating
Code Complexity	SonarQube	Cyclomatic Complexity, Cognitive Complexity

Static Code Test

Analisis kode statis dilakukan menggunakan SonarQube, yang mengevaluasi *maintainability rating* dan *complexity* dari kode aplikasi. *Maintainability rating* digunakan untuk menilai sejauh mana kode dapat dipelihara dan diperbaiki secara efisien. Sementara itu, *complexity* mencakup pengukuran seperti *cyclomatic complexity* dan *cognitive complexity*. Rangkuman pengujian ditunjukkan pada Tabel 3.10.

Load Testing

Dalam tahap *Load Testing*, fokus utama adalah mengevaluasi kinerja sistem dalam menangani lonjakan permintaan serta mengidentifikasi *bottleneck* yang dapat menghambat performa aplikasi (Godavarti, 2025). *Bottleneck* merujuk pada kondisi di mana suatu komponen sistem membatasi kapasitas keseluruhan, sehingga dapat menyebabkan perlambatan atau kegagalan saat menghadapi *traffic* tinggi. Untuk mengukur efektivitas sistem dalam skenario tersebut, digunakan tiga metrik utama: *response time*, yang mengukur kecepatan sistem dalam merespons permintaan (Thatikonda, dkk., 2024); *throughput*, yang menunjukkan jumlah permintaan yang dapat diproses dalam rentang waktu tertentu (Goncalves, dkk., 2021; Blinowski, dkk., 2022); serta *error rate*, yang mencerminkan proporsi permintaan yang gagal ditangani.

Pengujian dilakukan menggunakan Apache JMeter, dengan parameter variatif untuk merepresentasikan skenario pengguna nyata. Variabel yang digunakan antara lain jumlah pengguna virtual (*threads*) sebanyak 100, 500, dan 1000 (Freire, dkk., 2021; Sun, 2024; Thatikonda, dkk., 2024); durasi pengujian 0–30 menit; serta *ramp-up period* antara 10 hingga 60 detik. Setiap pengguna menjalankan *loop* atau iterasi sebanyak 1 hingga 10 kali. Iterasi tunggal diterapkan pada permintaan yang memerlukan data unik (misalnya proses *create*), sedangkan 10 iterasi digunakan untuk permintaan non-unik (seperti proses *read*) (Khusaeri, dkk., 2023).

Untuk memastikan pengujian berjalan konsisten dan terkontrol, digunakan *request body dataset* dalam format `.csv` yang dihasilkan secara otomatis menggunakan skrip Python. Dataset ini terdiri dari 1600 baris dan dibagi ke dalam tiga kelompok sesuai jumlah *threads*, sehingga setiap permintaan dalam skenario pengujian memiliki data yang unik. Pendekatan ini diterapkan secara identik pada sistem berbasis arsitektur *Monolith* dan *Modular Monolith* guna menjaga validitas perbandingan.

Terdapat dua jenis skenario pengujian: *consecutive request* dan *parallel request*. *Consecutive request* digunakan untuk menguji ketahanan sistem dalam menangani permintaan yang dikirim secara berurutan oleh masing-masing pengguna virtual. Sementara itu, *parallel request* bertujuan mengukur performa sistem ketika menerima permintaan secara simultan dari banyak pengguna dalam waktu bersamaan. Terdapat empat skenario paralel yang masing-masing diuji pada tiga konfigurasi *thread group* (100, 500, dan 1000) (Freire, dkk., 2021; Khusaeri, dkk., 2023).

Secara keseluruhan, pengujian ini bertujuan untuk mengevaluasi performa dan skalabilitas sistem dalam menghadapi beban tinggi, serta mengidentifikasi potensi *bottleneck*. Hasil pengujian akan digunakan untuk membandingkan efektivitas arsitektur *Modular Monolith* terhadap arsitektur *Monolith*

dalam konteks aplikasi berskala kecil seperti Tahsin App.

t-Test

Uji *t-test independent* digunakan untuk mengetahui apakah perbedaan hasil pengujian antara arsitektur *monolith* dan *modular monolith* benar-benar signifikan secara statistik, atau hanya disebabkan oleh variasi acak. Dalam penelitian ini, pengujian dilakukan sebanyak tiga kali untuk masing-masing arsitektur ($n = 3$ per kelompok). Uji dilakukan terhadap setiap *metric* performa (misalnya *throughput*, *error rate*, atau waktu respons), dengan asumsi bahwa data berasal dari dua populasi yang tidak saling bergantung. Pengujian tidak dilakukan di keseluruhan skenario pengujian, melainkan pada skenario pengujian yang memiliki nilai *metric* yang bervariasi. Tingkat signifikansi (*alpha*) ditetapkan sebesar $\alpha = 0,05$ (Yu, dkk., 2022), yang berarti peluang kesalahan dalam menyimpulkan adanya perbedaan adalah 5%. Proses perhitungan dilakukan menggunakan pustaka *scipy.stats* dalam bahasa pemrograman Python, yang menyediakan fungsi `ttest_ind()` untuk uji *t-test* dua sampel independen.

3.4 Analysis

Secara umum, proses analisis pada penelitian ini dilakukan dengan dua pendekatan. Pertama, menggunakan metode analisis khusus yang disesuaikan dengan masing-masing aspek kualitas perangkat lunak, seperti *maintainability*, *complexity*, *performance*, dan *scalability*. Kedua, hasil pengujian dan evaluasi tersebut kemudian dibandingkan dengan prinsip-prinsip desain arsitektur yang relevan serta temuan dari penelitian-penelitian sebelumnya. Pendekatan ini bertujuan untuk memastikan bahwa interpretasi hasil tidak hanya berdasarkan data numerik, tetapi juga didukung oleh kerangka teoritis dan pengalaman empiris yang telah ada.

3.4.1 Design Implementation

Penelitian ini bertujuan mengevaluasi penerapan *Hexagonal Architecture* dalam pendekatan *modular monolith*. Untuk itu, dilakukan analisis terhadap aspek desain perangkat lunak dengan pendekatan dan alat seperti pada Tabel 3.11.

Tabel 3.11. Langkah Analisis Implementasi Desain

Aspek yang Dianalisis	Metode atau Alat Analisis
Struktur folder	Ditinjau berdasarkan prinsip modularisasi berbasis fitur (<i>feature-based structure</i>) sesuai domain aplikasi
Arah dan pola dependensi antar <i>file</i>	Analisis <i>circular dependency</i> menggunakan <i>madge</i> untuk mengidentifikasi arah dan kompleksitas dependensi antar modul
Hubungan antar lapisan (<i>Controller – Service – Persistence</i>)	Observasi langsung terhadap alur eksekusi dan pemisahan tanggung jawab antar lapisan
Peran <i>framework</i> NestJS	Studi terhadap fitur bawaan seperti <i>dependency injection</i> , <i>module system</i> , dan struktur <i>file</i> default yang mendukung arsitektur modular
Perbandingan struktur <i>monolith</i> vs <i>modular monolith</i>	Analisis komparatif terhadap struktur dan dependensi dua versi aplikasi yang dibangun dalam studi kasus

3.4.2 Maintainability

Setelah memperoleh hasil pengukuran dari SonarQube, dilakukan analisis untuk membandingkan tingkat *maintainability* antara arsitektur *monolith* dan *modular monolith*. Fokus analisis berada pada interpretasi hasil numerik dan kategorikal yang dihasilkan oleh SonarQube. Langkah-langkah analisis dirangkum dalam Tabel 3.12.

Tabel 3.12. Langkah Analisis Hasil *Maintainability*

Langkah Analisis	Deskripsi
Membandingkan <i>maintainability rating</i> antar arsitektur	Menilai dampak modularisasi terhadap tingkat kemudahan pemeliharaan berdasarkan kategori <i>rating</i> yang diberikan (A–E)
Menganalisis rasio <i>technical debt</i>	Mengamati besarnya beban teknis relatif terhadap ukuran basis kode untuk mengevaluasi efisiensi dan keberlanjutan pengembangan
Mengamati jumlah dan jenis <i>issues</i>	Mengkaji apakah modularisasi berhasil mengurangi jumlah atau jenis permasalahan umum yang menghambat <i>maintainability</i>

3.4.3 Complexity

Setelah memperoleh hasil pengukuran dari SonarQube, dilakukan analisis untuk menilai dampak modularisasi terhadap tingkat kompleksitas kode. Penilaian dilakukan terhadap nilai *cyclomatic complexity* dan *cognitive complexity* dalam masing-masing versi arsitektur. Langkah-langkah analisis dirangkum pada Tabel 3.13.

Tabel 3.13. Langkah Analisis Hasil Complexity

Langkah Analisis	Deskripsi
Membandingkan nilai <i>cyclomatic complexity</i> antara arsitektur <i>monolith</i> dan <i>modular monolith</i>	Menganalisis seberapa besar peningkatan atau penurunan kompleksitas logika kontrol akibat modularisasi
Membandingkan nilai <i>cognitive complexity</i> antara arsitektur <i>monolith</i> dan <i>modular monolith</i>	Mengamati tingkat kesulitan pembacaan dan pemahaman kode berdasarkan hasil dari masing-masing versi arsitektur

3.4.4 Performance

Setelah dilakukan *load testing* menggunakan JMeter, analisis dilakukan untuk membandingkan performa aplikasi pada arsitektur *monolith* dan *modular monolith*. Fokus utama analisis adalah interpretasi hasil kuantitatif dari JMeter berdasarkan beberapa metrik kinerja. Langkah-langkah analisis dijabarkan pada Tabel 3.14.

Tabel 3.14. Langkah Analisis Hasil Performa

Langkah Analisis	Deskripsi
Membandingkan nilai <i>average response time</i> antar versi	Mengkaji seberapa cepat API merespons permintaan dalam masing-masing arsitektur
Membandingkan nilai <i>throughput</i> antar versi	Mengamati jumlah permintaan yang dapat ditangani per detik sebagai indikator efisiensi
Meninjau perubahan fluktuatif dari <i>error rate</i>	Mencari tahu pada skenario mana <i>error rate</i> mengalami kenaikan drastis untuk mengidentifikasi titik terjadinya <i>bottleneck</i>

3.4.5 Scalability

Setelah dilakukan pengujian skalabilitas menggunakan JMeter dengan berbagai skenario beban pengguna, dilakukan analisis untuk mengevaluasi apakah modularisasi memberikan dampak terhadap kemampuan sistem dalam menangani pertambahan beban tanpa degradasi performa. Selain itu juga untuk melihat bagaimana server merespons terhadap peningkatan kapasitas server (*vertical scaling*). Langkah-langkah analisis dijabarkan pada Tabel 3.15.

Tabel 3.15. Langkah Analisis Hasil *Scalability*

Langkah Analisis	Deskripsi
Membandingkan performa pada tiap skenario jumlah pengguna	Mengamati tren perubahan <i>response time</i> , <i>throughput</i> , dan <i>error rate</i> saat beban meningkat (misalnya dari 100 ke 1000 pengguna)
Membandingkan performa pada dua server berbeda	Mengevaluasi dampak penambahan kapasitas server (<i>vertical scaling</i>) terhadap performa masing-masing arsitektur, serta mengkaji apakah modularisasi lebih responsif terhadap peningkatan sumber daya
Mengaitkan hasil pengujian dengan struktur modular	Mengkaji apakah modularisasi membantu isolasi beban pada modul tertentu dan mengurangi efek domino dari lonjakan trafik

3.4.6 *Statistic Analysis*

Setelah memperoleh hasil uji statistik menggunakan *independent t-test*, analisis dilakukan untuk menentukan apakah terdapat perbedaan signifikan antara dua arsitektur. Langkah-langkah analisis dijelaskan pada Tabel 3.16.

Tabel 3.16. Langkah Analisis Statistik

Langkah Analisis	Deskripsi
Mengamati nilai p dari hasil <i>t-test</i>	Menentukan apakah nilai p lebih kecil dari nilai signifikansi (α) yang telah ditetapkan di bagian pengujian
Menentukan keberlakuan hipotesis	Jika $p < \alpha$, maka hipotesis nol (H_0) ditolak dan disimpulkan bahwa terdapat perbedaan yang signifikan antara dua kelompok
Menganalisis arah perbedaan	Mengidentifikasi kelompok mana yang memiliki nilai rata-rata lebih baik berdasarkan metrik yang diuji (misalnya, lebih rendah untuk <i>response time</i> , lebih tinggi untuk <i>throughput</i>)
Mengaitkan dengan desain arsitektur	Menjelaskan apakah perbedaan yang ditemukan selaras dengan karakteristik modularisasi, seperti isolasi modul, efisiensi proses, atau pengurangan ketergantungan